01. JavaScript y el DOM

Resumen de tus comentarios y apuntes del repositorio, revisado el 10 de noviembre de 2026. Ejemplos breves para consultar mientras programas; no constituyen una implementación completa ni garantizan que todos los pendientes estén resueltos.

Encontrar elementos

document representa la página. **getElementById()** busca por ID; **querySelector()** devuelve la primera coincidencia de un selector CSS; **querySelectorAll()** devuelve una colección de coincidencias.

```
const concepto = document.getElementById("concepto")
const cuerpoTabla = document.querySelector("tbody")
const filtros = document.querySelectorAll(".btn-filtro")
```

Si no existe el elemento, las dos primeras búsquedas devuelven **null**. Un elemento dentro de un comentario HTML no forma parte del DOM: no puedes añadirle eventos.

Leer y cambiar la interfaz

value lee o asigna el valor de un campo. **textContent** cambia su texto. **setAttribute()** modifica atributos; el valor se convierte a texto. **dataset.filtro** lee el atributo data-filtro. **classList.add()** añade una clase CSS.

Crear y colocar elementos

createElement() crea un nodo todavía fuera de la página; **append()** lo incorpora a un contenedor. **replaceChildren()**, sin argumentos, elimina sus hijos. **new Option(texto, valor)** crea una opción y **select.add()** la incorpora al selector.

```
const celda = document.createElement("td")
celda.textContent = "Transporte"
fila.append(celda)
```

En Git Money: el formulario y la tabla se construyen o actualizan a partir de los datos en memoria. Los encabezados de secciones en CSS se mantienen porque ayudan a localizar estilos.

02. Eventos y formularios

Un evento indica que algo ha ocurrido: un clic, una tecla o el envío de un formulario. La función registrada se ejecuta cuando sucede.

Escuchar una acción

```
boton.addEventListener("click", function(evento) {
    console.log(evento.target)
})
```

evento.target es el elemento donde se originó la acción; puede ser un hijo del elemento que tiene el listener. **evento.key** indica la tecla, por ejemplo "Escape".

Controlar un formulario

El evento **submit** permite recoger los campos. **preventDefault()** evita el envío normal del navegador para procesarlo con JavaScript. **reset()** restablece los valores iniciales; por eso la fecha y hora se asignan después.

```
formulario.addEventListener("submit", function(evento) {
    evento.preventDefault()
    const texto = concepto.value.trim()
})
```

trim() elimina espacios al principio y al final. **Number()** convierte un valor a número; los campos se leen normalmente como texto. Una conversión no sustituye la validación de los datos.

Diálogos y menú

showModal() abre un elemento dialog como ventana modal; **close()** lo cierra. **hidden** controla la ocultación de un elemento, **contains()** permite comprobar si un nodo está dentro de otro y **focus()** devuelve el foco a un control.

aria-expanded comunica si un panel está desplegado y **aria-pressed** si un botón de filtro está activado. Estos atributos expresan estado; la lógica y los estilos deben mantenerlo coherente.

Menú aparcado: sus constantes y eventos se conservan desactivados junto con su HTML. Reactivar solo una parte puede causar errores por referencias inexistentes.

03. Arrays, fechas e importes

El estado de la aplicación contiene categorías, movimientos, el tipo de movimiento del formulario y el filtro seleccionado.

Trabajar con listas

push() añade un elemento al array. **slice()**, sin argumentos, crea una copia superficial. **filter()** devuelve todos los elementos que cumplen una condición; **find()** devuelve el primero o undefined. Comprueba el resultado de find antes de leer sus propiedades.

```
const gastos = movimientos.filter(function(m) {
    return m.tipo === "gasto"
})
```

sort() modifica el array sobre el que se ejecuta. Su comparador devuelve un número negativo, positivo o cero para decidir el orden. En Git Money se ordena una copia por fecha y hora descendentes.

Fecha y hora locales

new Date() representa un instante. getFullYear(), getDate(), getHours() y getMinutes() obtienen sus partes locales. **getMonth()** devuelve de 0 a 11: se suma 1 para obtener el mes habitual. **String()** convierte a texto y **padStart(2, "0")** completa con ceros.

```
const ahora = new Date()
const mes = String(ahora.getMonth() + 1).padStart(2, "0")
const partes = "2026-11-10".split("-")
// partes: ["2026", "11", "10"]
```

Con campos de anchura fija, AAAA-MM-DD y HH:MM permiten comparar cronológicamente sus textos concatenados. Para mostrar la fecha se reorganizan sus partes como DD/MM/AAAA.

Cálculos y formato

toFixed(2) devuelve un **texto** con dos decimales, no un número. Úsalo para mostrar resultados, después de calcular. El operador + puede concatenar textos: normaliza los importes del servidor antes de sumar.

En Git Money: ingresos y gastos se calculan para el mes actual; el balance recorre todo el historial. El código actual no incluye el saldo inicial de la cuenta en ese balance.

04. HTTP, JSON y tu API

La API es el conjunto de puntos de entrada del servidor que reciben peticiones y devuelven respuestas. fetch es el cliente que los llama; no crea la API.

Pedir datos al servidor

```
const respuesta = await fetch("php/listar-categorias.php")
if (!respuesta.ok) {
    throw new Error("No se pudieron cargar las categorías")
}
const datos = await respuesta.json()
```

async hace que una función devuelva una promesa y permite usar await. **await** espera su resultado dentro de esa función; no bloquea toda la página. **try/catch** permite capturar errores, incluidos los de promesas esperadas con await.

fetch() no rechaza automáticamente una respuesta HTTP 400 o 500. **respuesta.ok** indica un estado entre 200 y 299. **respuesta.json()** lee el cuerpo e interpreta JSON; puede fallar si recibe HTML o JSON incorrecto.

Enviar un objeto

```
await fetch("php/guardar-movimiento.php", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify(movimiento)
})
```

GET consulta datos; **POST** los envía para su procesamiento. **JSON.stringify()** convierte el objeto en texto JSON. Content-Type describe el cuerpo enviado; no activa la validación ni guarda los datos por sí mismo.

Los endpoints de Git Money

listar-categorias.php: categorías. **listar-movimientos.php:** historial. **guardar-movimiento.php:** inserción. **estado.php:** diagnóstico de conexión. La ruta de cada petición identifica un endpoint.

Pendiente funcional: llamar a una función async sin await no espera su finalización. El flujo actual todavía debe coordinar las cargas y esperar la confirmación de guardado antes de tratar el movimiento como persistido.

05. PHP: peticiones y validación

PHP se ejecuta en el servidor. Recibe datos del navegador, aplica reglas y responde. Sus variables comienzan por $.

Entrada y salida JSON

```
header("Content-Type: application/json; charset=utf-8");
$metodo = $_SERVER["REQUEST_METHOD"];
$cuerpo = file_get_contents("php://input");
$datos = json_decode($cuerpo, true);
```

header() declara el formato de respuesta y debe enviarse antes de producir contenido de salida. **php://input** permite leer el cuerpo de la petición. **json_decode(..., true)** interpreta un objeto JSON como un array asociativo; **json_encode()** prepara la respuesta JSON.

Comprobar antes de guardar

json_last_error() informa del último error JSON; se compara con JSON_ERROR_NONE. **isset()** comprueba que una clave exista y no sea null. **is_numeric()** comprueba si el valor es numérico o un texto numérico; hay que exigir además un importe mayor que cero.

is_string() comprueba texto. **trim()** ayuda a rechazar conceptos que solo contienen espacios. **?? null** permite utilizar null cuando una clave no existe o ya es null. **exit** termina el script tras enviar el error.

Estados HTTP y errores

400: petición incorrecta. **405:** método no permitido. **500:** error del servidor. http_response_code() establece el estado; la respuesta JSON comunica el mensaje. Las comprobaciones HTML ayudan al usuario, pero no sustituyen las del servidor.

catch (Throwable $error) captura excepciones y otros errores de PHP. getMessage() obtiene el detalle; error_log() lo escribe en el destino de registro configurado, que puede ser la terminal o un archivo. Devuelve al cliente un mensaje comprensible sin exponer ese detalle interno.

06. PDO y conexión a PostgreSQL

PDO es una interfaz de PHP para acceder a bases de datos mediante sus controladores. En este proyecto se utiliza el de PostgreSQL.

Conexión y configuración

```
$dsn = "pgsql:host=$host;port=$puerto;dbname=$baseDatos";
$contrasenia = getenv("GIT_MONEY_DB_PASSWORD");
$conexion = new PDO($dsn, $usuario, $contrasenia);
```

El **DSN** identifica motor, servidor, puerto y base de datos. **getenv()** lee una variable del entorno del proceso PHP; puede heredarse de la terminal o configurarse en el servidor. Si no existe, devuelve false. El proyecto rechaza también una contraseña vacía.

Ejemplo para definirla en PowerShell antes de iniciar el servidor PHP en esa misma sesión:

```
$env:GIT_MONEY_DB_PASSWORD = Read-Host "Contraseña" -MaskInput
```

require_once carga un archivo una sola vez. __**DIR**__ es la carpeta del archivo PHP actual y permite construir una ruta estable a conexion.php.

Consultar y recuperar filas

query() ejecuta una consulta fija. **fetchAll(PDO::FETCH_ASSOC)** recupera filas como arrays asociativos cuyas claves son los nombres de las columnas. Los tipos que llegan al navegador deben comprobarse: un importe decimal puede recibirse como texto.

Guardar con parámetros

```
$consulta = $conexion->prepare(
    "INSERT INTO ejemplo (concepto) VALUES (:concepto)"
);
$consulta->execute(["concepto" => $concepto]);
```

prepare() prepara la sentencia y **execute()** vincula los valores a sus parámetros. Así no se concatena la entrada del usuario dentro del SQL. Los parámetros se utilizan para valores, no para sustituir arbitrariamente nombres de tablas o columnas.

Modelo actual: procedencia se usa en ingresos y medioPago en gastos; el otro campo debe ser NULL. Se conservan ambos, pero sus valores actuales son provisionales. Cuenta y usuario también están fijados en 1.

07. SQL: consulta rápida

Síntesis de apuntesSQL y del esquema del proyecto. Los archivos de pruebas son históricos: no son una instalación ni una migración lista para ejecutar.

Crear y restringir datos

CREATE DATABASE crea una base; **CREATE TABLE** crea una tabla. **SERIAL** y **GENERATED ALWAYS AS IDENTITY** permiten generar identificadores. **VARCHAR(n)** limita caracteres. **NUMERIC(12,2)** admite 12 cifras en total, dos decimales: hasta 10 cifras enteras.

PRIMARY KEY identifica cada fila; **NOT NULL** exige valor; **UNIQUE** evita duplicados; **CHECK** comprueba una condición; **REFERENCES** establece una clave foránea. Las claves compuestas de cuenta/categoría con usuario impiden asociar un movimiento a propietarios distintos.

Consultar y relacionar

```
SELECT m.id, c.nombre
FROM movimiento AS m
JOIN cuenta AS c ON m.idcuenta = c.id
WHERE m.idusuario = 1
ORDER BY m.fecha DESC, m.hora DESC, m.id DESC;
```

SELECT consulta; version() devuelve la versión de PostgreSQL. **WHERE** filtra; **JOIN ... ON** relaciona tablas; **AS** asigna un alias; **ORDER BY ... DESC** ordena de mayor a menor. Una selección de columnas concretas limita los datos devueltos.

Insertar, actualizar y modificar el esquema

INSERT INTO ... VALUES añade filas. Usa comillas simples para textos: las dobles delimitan identificadores en PostgreSQL. **UPDATE ... SET ... WHERE** modifica filas; sin WHERE afecta a todas. **ALTER TABLE** cambia tablas con ADD COLUMN, ALTER COLUMN o ADD CONSTRAINT.

TRUNCATE ... RESTART IDENTITY vacía tablas y reinicia sus secuencias asociadas; es una operación de pruebas que elimina datos. **ALTER ROLE ... PASSWORD** cambia la contraseña de un rol: no se deben copiar contraseñas reales en los apuntes.

Cómo usar estos apuntes

Consulta el tema que necesites, identifica su uso en el código y prueba un ejemplo aislado. No ejecutes los SQL antiguos en bloque: contienen campos desactualizados e instrucciones incompletas. El esquema versionado está en SQL/esquema.sql.

Fuentes: comentarios de js/script.js y los cinco archivos de php; apuntesSQL/apuntes.sql, modificaciones.sql y pruebas.sql; SQL/esquema.sql. Se han resumido repeticiones y corregido explicaciones, sin cambiar la lógica de la aplicación.